UNITÀ 2 · PROGETTARE E UTILIZZARE WEB SERVICE · LEZIONE 3

Le architetture orientate ai servizi

Dal grande programma unico a decine di servizi che collaborano: come si progettano i sistemi delle grandi aziende, e quando non conviene imitarle.

5G TPSI Monolite, SOA, ESB, microservizi, API gateway

PER COMINCIARE

Devi costruire un'app per le consegne a domicilio. Come la organizzi?

SONDAGGIO

- **A** Un unico programma, ben scritto
- **B** Tre o quattro grandi servizi
- **C** Decine di piccoli servizi indipendenti
- **D** Dipende da quante persone ci lavorano

IL PUNTO DI PARTENZA

Il monolite: tutto in un solo programma

Interfaccia, logica e accesso ai dati in un'unica applicazione, che si compila, si installa e si avvia tutta insieme, con un solo database.

Pregi

- Semplice da sviluppare e testare
- Chiamate tra moduli velocissime
- Una sola cosa da installare

Difetti, crescendo

- Ogni modifica richiede di ripubblicare tutto
- Un errore può fermare tutto
- Scala solo per intero

Applicazione unica

Catalogo · Carrello · Ordini · Pagamenti · Utenti

Un database

tutte le tabelle insieme

Monolite non è un insulto: molti servizi famosi sono partiti così, e alcuni lo sono ancora.

SOA · SERVICE-ORIENTED ARCHITECTURE

Il sistema come insieme di servizi

Un **servizio** è una funzione di business (pagare, spedire, autenticare) offerta in rete attraverso un contratto pubblico. Chi lo usa conosce il contratto, non il codice.

Contratto

Operazioni, formati dei dati ed errori descritti in modo formale (per esempio un WSDL).

Accoppiamento debole

Posso riscrivere un servizio senza cambiare gli altri, se il contratto resta uguale.

Riusabilità

Lo stesso servizio "Pagamenti" serve il sito, l'app e il negozio fisico.

PUZZLE

Quale principio sta rispettando?

Accoppiamento debole	Il servizio Catalogo funziona anche se Pagamenti è giù
Astrazione	Il servizio Spedizioni passa da Java a Go e nessuno se ne accorge
Autonomia	Un catalogo interno elenca tutti i servizi con i loro contratti
Componibilità	"Checkout" chiama in sequenza Carrello, Pagamenti e Spedizioni
Scopribilità	Il client non sa che dietro c'è un database Oracle

UNA STORIA VERA, RACCONTATA DA UN EX DIPENDENTE

Amazon, circa 2002: "solo interfacce di servizio"

Secondo il racconto dell'ingegnere **Steve Yegge** (2011), Jeff Bezos impose a tutti i team regole semplici e brutali:

1. Ogni team espone dati e funzioni solo tramite **interfacce di servizio**.
2. I team comunicano tra loro solo attraverso queste interfacce: niente accessi diretti ai database degli altri.
3. Ogni interfaccia va progettata come se dovesse essere usata da **clienti esterni**.

Il risultato

Se un servizio interno è già pronto per l'esterno, lo si può vendere. Nel 2006 nascono Amazon S3 ed EC2: l'inizio di **Amazon Web Services**, oggi il più grande fornitore di cloud.

ESB · ENTERPRISE SERVICE BUS

Il bus che collega tutti i servizi

Nella SOA classica i servizi non si chiamano direttamente: passano da un **bus** centrale che

- instrada i messaggi al servizio giusto
- trasforma i formati (un XML vecchio in uno nuovo)
- coordina processi lunghi (orchestrazione)
- applica sicurezza e registra il traffico

Rischio: il bus diventa intelligente, enorme e indispensabile. Un nuovo monolite, nel mezzo.

Orchestrazione

Un direttore d'orchestra (il servizio Checkout o l'ESB) dice a ciascuno cosa fare e in che ordine.

Coreografia

Nessun direttore: ogni servizio reagisce a eventi ("ordine pagato") e pubblica i suoi ("pacco spedito").

AL COMPUTER · 25 minuti · a coppie

Scomponi la mensa della scuola in servizi

1. Apri **app.diagrams.net**, scegli "Decide later" o "Dispositivo": non serve un account.
2. Requisiti: menu del giorno, prenotazione del pasto, pagamento con borsellino, avvisi ad allergici, statistiche per il cuoco.
3. Disegna da 4 a 6 servizi: per ognuno nome, 2 operazioni e i **dati che possiede**.
4. Collega con frecce chi chiama chi, scrivendo l'operazione sulla freccia.
5. Esporta come **mensa-soa-cognome.png**.

Controllo: ogni dato ha un solo proprietario (per esempio il saldo sta solo in Pagamenti) e nessun servizio legge il database di un altro. Se il servizio Statistiche cade, si può ancora prenotare.

MICROSERVIZI

SOA, in piccolo e senza bus

Nel 2014 James Lewis e Martin Fowler descrivono uno stile che molte aziende già usavano: un'applicazione come insieme di piccoli servizi, ognuno nel suo processo, che comunicano con meccanismi leggeri, spesso API HTTP.

- Ogni servizio ha **il suo database**.
- Si rilascia da solo, anche più volte al giorno.
- Lo possiede un team piccolo, dall'inizio alla produzione.
- Ogni team sceglie linguaggio e tecnologia.

Il conto da pagare

- Tante chiamate di rete: latenza e guasti (lezione 1)
- Niente transazioni tra servizi diversi
- Monitoraggio e log distribuiti
- Serve automazione: container, Kubernetes, CI/CD

MENTI CHE CI HANNO PRECEDUTO

La legge di Conway

> **Le organizzazioni che progettano sistemi sono costrette a produrre progetti che copiano la loro struttura di comunicazione.**
>
> **Melvin Conway**, informatico statunitense, in *How Do Committees Invent?*, Datamation, 1968

L'esempio di Conway: cinque persone sul compilatore COBOL e tre su quello ALGOL. Risultato: un compilatore in cinque fasi e uno in tre. Un'azienda con un team per ogni servizio produce microservizi.

CONFRONTO

Tre architetture a confronto

	Monolite	SOA classica	Microservizi
Unità	Un'applicazione	Servizi di business grandi	Servizi piccoli, uno per capacità
Comunicazione	Chiamate di metodo	ESB, spesso SOAP	HTTP/REST, gRPC, eventi
Dati	Un database	Spesso condivisi	Un database per servizio
Rilascio	Tutto insieme	Per servizio, coordinato	Indipendente, frequente
Quando	Team piccolo, dominio nuovo	Integrare sistemi aziendali esistenti	Molti team, carichi molto diversi

All'esame: non basta dire "uso i microservizi". Scrivi **perché** (carico, team, rilasci) e **che cosa paghi** (rete, coerenza, complessità).

QUIZ A TRABOCCHETTO

Quale affermazione sui microservizi è corretta?

QUIZ

A Sono sempre più veloci di un monolite

B Condividono un unico database per restare coerenti

C Ogni servizio dovrebbe possedere i propri dati

D Vanno scritti tutti nello stesso linguaggio

METTITI ALLA PROVA

Prime Video, 2023: uno strumento di monitoraggio passa da servizi distribuiti a un unico processo. Che cosa succede?

QUIZ

A Più affidabile, ma con costi molto più alti

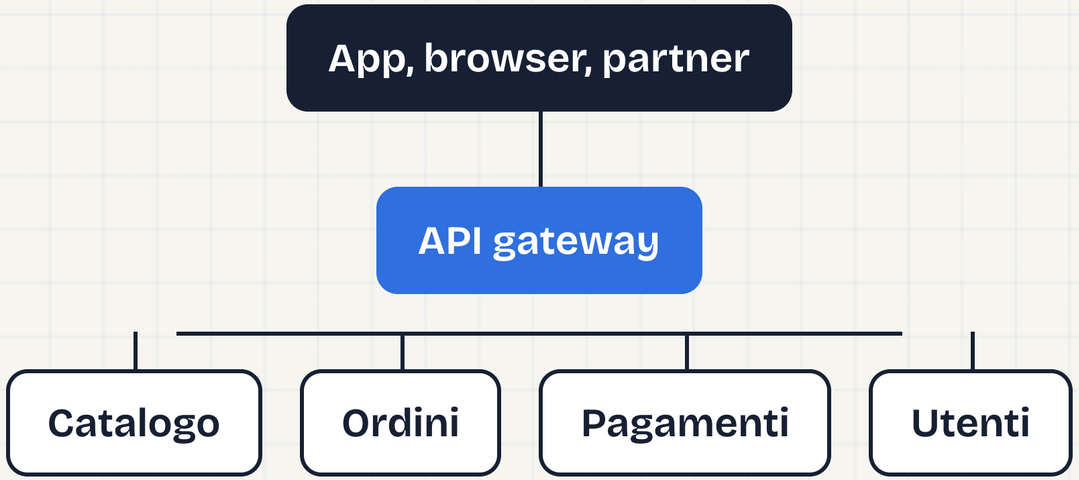
B Costi di infrastruttura ridotti di oltre il 90%

C Nessuna differenza misurabile

D Tornano ai servizi distribuiti dopo un mese

API GATEWAY

Una sola porta d'ingresso per tanti servizi



- **Instradamento**: `/ordini` va al servizio Ordini
- **Autenticazione**: controlla chiavi e token una volta sola
- **Limiti**: troppe richieste? `429`
- **TLS**, log, statistiche, versioni dell'API
- **Aggregazione**: una chiamata del client, più servizi dietro

Esempi: Amazon API Gateway, Azure API Management, Google Apigee, Kong, NGINX.

PUZZLE

Il viaggio di una richiesta attraverso il gateway

1. Il gateway controlla il limite di richieste: sotto la soglia
2. La richiesta viene inoltrata al servizio Ordini
3. Il gateway termina la connessione TLS
4. Il gateway verifica la chiave: valida
5. Ordini risponde e il gateway registra la chiamata nei log
6. L'app invia `GET /ordini/7` con la sua chiave API

AL COMPUTER · 15 minuti

Gateway, parte 1: due microservizi

```
import com.sun.net.httpserver.*;
import java.net.*;
import java.net.http.*;

public class Gateway {
  static void rispondi(HttpExchange ex, int code, byte[] b)
      throws java.io.IOException {
    ex.getResponseHeaders().set("Content-Type", "application/json");
    ex.sendResponseHeaders(code, b.length);
    ex.getResponseBody().write(b);
    ex.close();
  }
  static void servizio(int porta, String json) throws Exception {
    var s = HttpServer.create(new InetSocketAddress(porta), 0);
    s.createContext("/", ex -> rispondi(ex, 200, json.getBytes()));
    s.start();
  }
  // main nella parte 2
}
```

1. Crea **Gateway.java** con questi due metodi.
2. Il metodo `servizio` avvia un microservizio finto su una porta.
3. Nel `main` (parte 2) ne avvieremo due: **menu** sulla 9001 e **ordini** sulla 9002.

Controllo: per ora il file deve solo compilare: `javac Gateway.java` senza errori (manca il main, quindi non si avvia ancora).

AL COMPUTER · 25 minuti

Gateway, parte 2: instradare e proteggere

```
public static void main(String[] a) throws Exception {
  servizio(9001, "[{\"id\":1,\"piatto\":\"Lasagne\",\"prezzo\":6.5}]");
  servizio(9002, "[{\"id\":7,\"stato\":\"in preparazione\"}]");
  var http = HttpClient.newHttpClient();
  var gw = HttpServer.create(new InetSocketAddress(9000), 0);
  gw.createContext("/", ex -> {
    String path = ex.getRequestURI().getPath();
    int porta = path.startsWith("/menu") ? 9001
              : path.startsWith("/ordini") ? 9002 : 0;
    if (!"5G".equals(ex.getRequestHeaders().getFirst("X-Api-Key")))
      rispondi(ex, 401, "{\"errore\":\"chiave mancante\"}".getBytes());
    else if (porta == 0)
      rispondi(ex, 404, "{\"errore\":\"servizio sconosciuto\"}".getBytes());
    else try {
      var r = http.send(HttpRequest.newBuilder(URI.create(
          "http://localhost:" + porta + path)).build(),
          HttpResponse.BodyHandlers.ofByteArray());
      rispondi(ex, r.statusCode(), r.body());
    } catch (Exception e) {
      rispondi(ex, 502, "{\"errore\":\"servizio giu\"}".getBytes());
    }
  });
  gw.start();
}
```

1. Aggiungi il `main` e avvia: `java Gateway.java`
2. In un altro terminale: `curl -i localhost:9000/menu`
3. `curl -H "X-Api-Key: 5G" localhost:9000/menu`
4. Prova anche `/ordini` e `/pagamenti`.

Controllo: senza chiave **401 Unauthorized**; con la chiave il JSON delle lasagne; `/pagamenti` dà **404**.

OGGI

Servizi, gateway e intelligenza artificiale





BIGLIETTO D'USCITA

Completa prima di uscire

Nella SOA ciò che un servizio offre agli altri è descritto da un _____.

Il bus centrale della SOA classica si chiama _____.

Nei microservizi ogni servizio possiede il proprio _____.

Il punto unico d'ingresso che instrada le richieste ai servizi è l'API _____.

Secondo la legge di _____, i sistemi copiano la struttura di comunicazione di chi li progetta.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- Monolite: semplice, ma cresce male
- SOA: servizi con contratto, accoppiamento debole, ESB
- Microservizi: piccoli, un database ciascuno, rilasci indipendenti
- Più servizi = più rete, più guasti, più automazione
- API gateway: instrada, autentica, limita, registra
- La scelta giusta dipende da carico, team e dati

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.